



JavaScript

講義資料 - 後期

今回の内容 - 教科書準拠

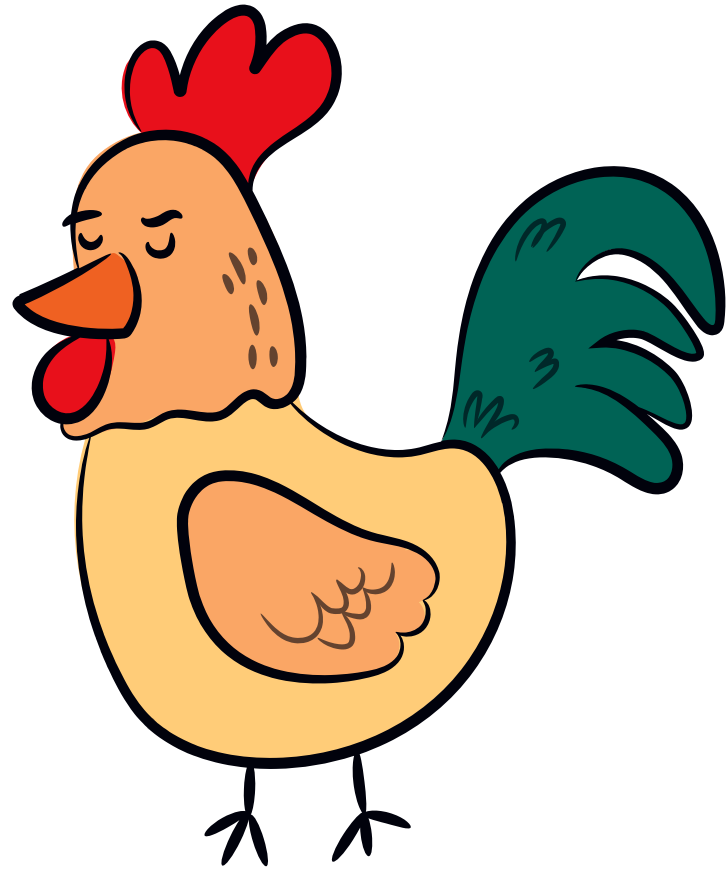
- 1.1-1 クラスとは？
- 2.1-2 クラスの使い方
- 3.1-3 静的なプロパティ
- 4.1-4 クラス継承
- 5.1-5 コードを実装してみよう

1-1 クラスとは？

クラスの考え方

複数の変数と関数(メソッド)で構成されるプログラムのかたまり

クラスに属する関数をメソッド、変数をフィールドと表現する。
実体である「オブジェクト」に対し、「クラス」は設計書に近いものとも言われる。



- 名前は？
- 鳴き声は？
- どう動く？

…動物として共通して持っている特徴である。

しかし、それぞれの動物によって違いがある。

∴「名前」、「鳴き声」、「動き方」という定義(設計書)を用意しておき、具体的なデータは必要な時に後で割り当てる。

1-2 クラスの使い方

クラスの宣言方法

通常のカラス宣言

```
class Sample {  
    // ここにカラスの宣言  
}
```

カラス式型の宣言方法

```
const Sample2 = class ClassName {  
    // ここに宣言  
}
```

講義内では、通常のカラス宣言のみを使用する。
「late/chapter1/sample2.html」を参照。

1-2 クラスの使い方

プロパティを定義する

フィールドとメソッドの定義方法

```
class Sample3 {  
    varSample;  
    methodSample() {  
        // メソッドの処理  
    }  
}
```

コンストラクタの使い方

「コンストラクタ」とは、実体を宣言した時に実行される特別なメソッド。
初期化や初期設定など、何かしら最初にやっておく必要のある場合に実行させる。

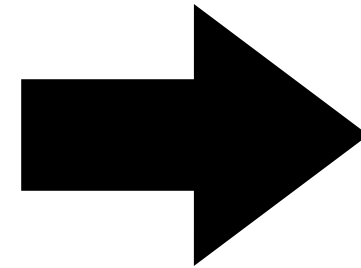
```
class Sample3 {  
    constructor(value1) {  
        // ここにコンストラクタの初期化処理  
    }  
}
```


クラスの仕組みを理解しよう

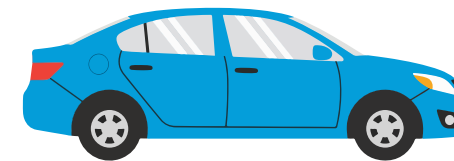
■車クラス(設計図)

- ー走るメソッド
- ー値引きメソッド
- ー名前フィールド
- ー重さフィールド

プロパティ



具体的なデータとして
実体化
(インスタンス)



名前: ガソリン車

重さ: 300kg

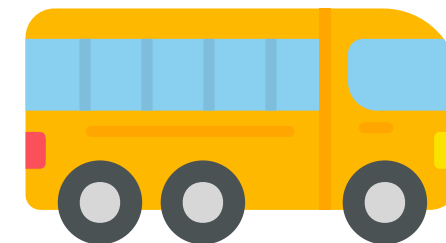
走るメソッド: 形と重さに依存



名前: トラック

重さ: 500kg

走るメソッド: 形と重さに依存



名前: バス

重さ: 1,000kg

走るメソッド: 形と重さに依存

フィールドとメソッドの定義方法

変数名 = new クラス名(コンストラクタへの引数);

コンストラクタがない場合と、コンストラクタに引数がない場合も空欄で()はつける。

【記述例】

```
const classIns = new Sample();
```

1-2 クラスの使い方

publicとprivateなプロパティ

公開による宣言(public: パブリック)とは？

実体から直接アクセス可能なプロパティのこと。

通常の変数・関数のように宣言した場合にデフォルトで適用される。

private含め比較的最近できた仕様のため、プロジェクトによっては使い分けがされていない可能性あり(ES2022)。

非公開による宣言(private: プライベート)とは？

実体からの直接アクセスができないプロパティのこと。

具体的に使う場合は、クラス内の他のメソッドなどから呼ばれた時。

外部から直接使う必要のない場合にはこちらで宣言する。

変数をprivateで、メソッドをpublic管理するのが「オブジェクト指向」では一般的な使い方。

↓ privateでは頭に「#」をつける

```
#fieldName;
```

```
#privateMethod() {}
```

1-2 クラスの使い方

例題

「sample2.html」を参考に「example2.html」をやってみよう。

記述内容はコメントで指示しているので、それにしたがって記入する。

1-3 静的なプロパティ

静的とは？

動的な使い方との違いを理解する

静的な使い方の場合は、クラスを実体化せずに直接扱うことができる。

※new の実体化文が不要。

一方で実体を通さず使うためクラス内で共通となっており、実体別の設定はできない。

✓ サンプルで言うと、犬とニワトリのような分け方が出来なくなる。

詳細は「sample3.html」を見てみよう。

【記述方法】

宣言の前に「static」を付ける

```
static name;
```

```
static methodName() {}
```

呼び出す時は、クラス名でアクセスできる。

```
ClassName.name;
```

```
ClassName.methodName();
```

1-3 静的なプロパティ

静的プロパティと実体

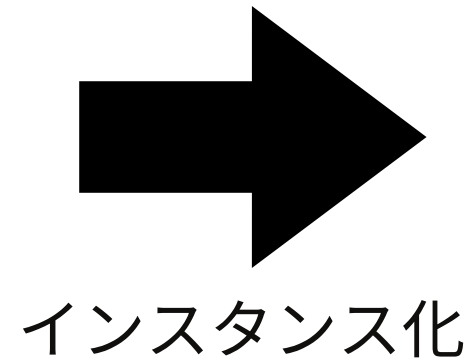
■日本車クラス(設計図)

■静的プロパティ(実体)

- ーstatic 走るメソッド
- ーstatic 生産国

■プロパティ(設計図のみ)

- ー社名表示メソッド
- ー名前フィールド

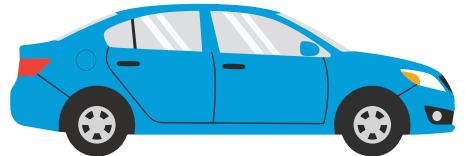


ーメソッド: T社製ですと表示

ー名前: T社

×走るメソッドは直接呼べない

×生産国フィールドは直接呼べない



ーメソッド: MS社製ですと表示

ー名前: MS社

×走るメソッドは直接呼べない

×生産国フィールドは直接呼べない



■静的のフィールドを呼ぶ場合

ー日本車クラス.走るメソッド

ー日本車クラス.生産国

∴クラスの設計書から直接実体を呼ぶ、単一のため値はクラス内で共通になる

1-3 静的なプロパティ

例題

「sample3.html」を参考に「example3.html」をやってみよう。

記述内容はコメントで指示しているので、それにしたがって記入する。

1-4 クラス継承

継承とは？

親クラスの情報を引き継ぐ= 継承する

設計書としてクラスを定義するが、似ている別物などがあったらどうするか。

- コピペして使う…
- 別物として新しく作成する…

そのような効率の悪さを解決するための機能が継承。

継承の具体的な動作とは？

親クラスのプロパティを利用することができる。

つまり、継承先の子クラスでは親クラスで不足したものを追加したり、上書きしたりして設計をし直すのが役割。

「sample4.html」を参照。

【記入例】

```
class childClass extends parentClass () {}
```

継承を図解してみよう

■Animalクラス

- constructor
- name(フィールド): 名前
- sound(メソッド): 鳴き声

fishクラスの「swim」は呼べない

仕様上メソッドについては、Javascriptのプロトタイプ継承というものが行われる。

継承

■Catクラス

- super(親コンストラクタ)
- constructor
- (継)name(フィールド): 名前
- (継)sound(メソッド): 鳴き声

fishクラスの「swim」は呼べない

■Dogクラス

- super(親コンストラクタ)
- constructor
- (継)name(フィールド): 名前
- (再定義)sound(メソッド): 鳴き声

fishクラスの「swim」は呼べない
親クラスの「sound」は呼べない

■Fishクラス

- super(親コンストラクタ)
- constructor
- (継)name(フィールド): 名前
- (継)sound(メソッド): 鳴き声
- swim(メソッド): 泳ぐ

1-4 コードを実装してみよう

演習

演習を通してクラスの記述に慣れよう。

最低限の情報とコメントを頼りに、コードを実装してみよう。
演習は複数あるので、「example-4-連番」になっている。

1-5 コードを実装してみよう

演習

単元のまとめとして「example5.html」の演習をしてみよう

最低限の情報とコメントを頼りに、コードを実装してみよう。

↓ コンソールの例

認証しました

トヨタのヤリスが走る。

トヨタのヤリスを運転する。

リチウムイオン搭載の日産のエリアが走る。

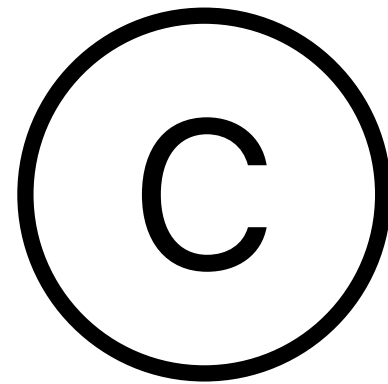
日産のエリアを運転する。

いすずのエルフが走る。

いすずのエルフを運転する。

積荷は食品です。

今回のまとめ



クラスの役割を理解する

ライブラリなどはクラスで定義されることは珍しくない。より複雑なモジュール処理を理解するためにも、クラスの基本的な使い方を理解しよう。



JavaScript

【改変】 実習 ファイルを分割して読み込んでみよう

今回の内容 - 教科書準拠

1.2-1 import/exportでファイルを分割しよう

2-1 import/exportでファイルを分割しよう

import/export

ファイルの分割と読み込みを実現する

外部ファイルに公開する場合は「export」、取り込む場合は「import」で宣言する。

✔ module機能を使うとセキュリティーモードになり、ローカルサーバー(LiveServer)を立てる必要がある。

- HTMLから読み込むときは、scriptの「type」を「module」にする
- スcopeが分かれるため、importした場所でロジックを書く

ファイル分割のメリットとは？

- 外部ライブラリなどを使うとき、ファイルをいくつも読み込む必要がない
- node.jsを始めとする開発環境を使う場合と考え方が似ている
 - →今後の講義の下地になるということ

2-1 import/exportでファイルを分割しよう

演習

サンプルや演習を通じて定着しよう

- 新しい構文を以下から確認しよう「late/chapter2/sup1/index.html」
- サンプルを確認してみよう「late/chapter2/sample1/index.html」
- 演習を試してみよう「late/chapter2/example1/index.html」